Lecture 22: Multipurpose Internet Mail Extension, MIME
(RFCs *2045*, *2046*, *2047*, *2048*, *2049*)

Objectives:

- Learn about the MIME format
- Learn how to write a basic MIME parser

1. Overview of MIME

The original SMTP protocol (RFC 821) and Message Format (RFC 822) were designed to construct mail and send it using only text in ASCII encoding (7 bits).

This was soon found to be very limited since it cannot be used to send binary data or even text data in non-ASCII format.

The Multipurpose Internet Mail Extension (MIME) was designed to extend the capability of SMTP mail but without violating the SMTP standard itself.

That is, the mail is still sent using only ASCII text, but other types of data can be sent by first encoding them into ASCII characters and appending them to the mail.

Some common encoding methods are **UUEncoding** and **BASE64** and **quoted-printable**.

The MIME protocol provides additional headers that are included in the message body to identify the different parts a mime-message.

There are six important headers that were introduced to identify MIME messages as follows.

- MIME-Version
- Content-Type
- Content-Transfer-Encoding
- Content-Disposition
- Content-ID
- Content-Description

1.1 MIME-Version:

This is a required header indicating that this message is composed using the MIME protocol.

MIME-Version: 1.0 is the only currently defined MIME-Version header allowed.

The MIME-Version header is a top-level header and does not appear in body parts unless the body part is itself an encapsulated fully formed message of content-type: message/rfc822, which might have its own MIME-Version header.

1.2 Content-Type:

This header is used to specify the media (data) type and subtype in the body of a message and to fully specify the representation of such data.

The simple form of this header is: Content-type: type/subtype
e.g: Content-type: image/gif
Content-type: text/plain; charset="iso-8859-1"
Content-type:text/html; charset="iso-8859-1"
Content-type: application/msword

There are seven main types defined, namely: text, image, audio, video, application, multipart and message. A number of sub-types are defined under each of these categories.

An email message may contain more than one of these simple content types at the same time. In that case, at the top of the document, the Content-type: multipart/mixed is used. The format is:

Content-type: multipart/mixed; boundary="uniqueBoundary"

The body of the message is then divided along the "uniqueBoundary" where each simple content-type is preceded by :
--uniqueBoundary.

Following each boundary, the content type of the part represented in that boundary is specified using the simple format: Content-type: type/subtype

Other headers particular to the part are also specified.

The headers are then followed by a blank line and then the body of the part.

The end of the multipart document is indicated by:
--uniqueBounday**--**

An email message may also have alternative parts, where the MUA is expected to select one of the options. In that case, the multipart/alternative is used. The format is:

Content-type: multipart/alternative; boundary=”anotherUniqueBoundary”

The “anotherUniqueBoundary” is used to separate between the different alternatives in a similar manner to multipart/mixed.

It is also possible to have both multipart/mixed and multipart/alternative contents at the same time in a nested manner. In this case, the boundaries must be different.

1.3 Content-Transfer-Encoding:

The Content-Transfer-Encoding header describes what encoding is used for a particular part of the message body.
e.g.: Content-Transfer-Encoding: base64

If a part does not have a Content-Transfer-Encoding header, the content transfer encoding of the part is assumed to be ASCII.

1.4 Content-Disposition:

This is used to provide information about how to present a message or a body part. The options are **inline** or **attachment**.

A bodypart should be marked `**inline'** if it is intended to be displayed automatically upon display of the message.

Bodyparts can be designated `**attachment'** to indicate that they are separate from the main body of the mail message, and that their display should not be automatic.

When a body part is to be treated as an attached file, the Content-Disposition header will include a file name parameter.
e.g: Content-Disposition: attachment; filename="saudiflag.gif"

1.5 Content-ID:

Content-ID headers are unique values that identify body parts, individually or as groups. They are necessary at times to distinguish body parts and allow cross-referencing between body parts.

1.6 **Content-Description**:

This is used to add descriptive text to non-textual body parts.

The following shows a sample of a MIME-message.

```
Received: from khuzama.ccse.kfupm.edu.sa (localhost [127.0.0.1])
        by khuzama.ccse.kfupm.edu.sa (8.11.0/8.9.3) with ESMTP id h4J8QuH07469
        for <bmghandi@ccse.kfupm.edu.sa>; Mon, 19 May 2003 11:26:56 +0300 (Saudi Stand
Time)
//deleted
Received: from soldier.ccse.kfupm.edu.sa(196.1.64.147) by ccsevs.ccse.kfupm.edu.sa via csm
        id 28329; Mon, 19 May 2003 08:29:47 +0000 (UTC)
Received: from icsbmghandi (ics-bmghandi.pc.ccse.kfupm.edu.sa [196.1.65.143])
        (authenticated bmghandi (0 bits))
        by soldier.ccse.kfupm.edu.sa (8.11.1/8.11.1) with ESMTP id h4J8P6D00647
        for <bmghandi@ccse.kfupm.edu.sa>; Mon, 19 May 2003 11:25:06 +0300 (Saudi Stand
Time)
Message-ID: <002f01c31de0$57ce6610$8f4101c4@icsbmghandi>
From: "Bashir Mohammed Ghandi" <bmghandi@ccse.kfupm.edu.sa>
To: "Bashir Mohammed Ghandi" <bmghandi@ccse.kfupm.edu.sa>
Subject: Testing Attachement I
Date: Mon, 19 May 2003 11:26:27 +0300
MIME-Version: 1.0
X-Priority: 3
X-MSMail-Priority: Normal
X-Mailer: Microsoft Outlook Express 6.00.2800.1106
X-MimeOLE: Produced By Microsoft MimeOLE V6.00.2800.1106
Content-Type: multipart/mixed;
        boundary="----=_NextPart_000_002B_01C31DF9.7CF57870"
Content-Length: 137705

This is a multi-part message in MIME format.

------=_NextPart_000_002B_01C31DF9.7CF57870
Content-Type: multipart/alternative;
        boundary="----=_NextPart_001_002C_01C31DF9.7CF57870"
```

------=_NextPart_001_002C_01C31DF9.7CF57870
Content-Type: text/plain;
charset="iso-8859-1"
Content-Transfer-Encoding: quoted-printable

Salaam,

This is testing attachement.
Please ignore.

Regards,
Bashir
------=_NextPart_001_002C_01C31DF9.7CF57870
Content-Type: text/html;
charset="iso-8859-1"
Content-Transfer-Encoding: quoted-printable

<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.0 Transitional//EN">
<HTML><HEAD>
<META http-equiv=3DContent-Type content=3D"text/html; =
charset=3Diso-8859-1">
<META content=3D"MSHTML 6.00.2800.1141" name=3DGENERATOR>
<STYLE></STYLE>
</HEAD>
<BODY bgColor=3D#ffffff>
<DIV><FONT face=3DArial size=3D2>Salaam,</FONT></DIV>
<DIV><FONT face=3DArial size=3D2></FONT> </DIV>
<DIV><FONT face=3DArial size=3D2>This is testing =
attachement.</FONT></DIV>
<DIV><FONT face=3DArial size=3D2>Please ignore.</FONT></DIV>
<DIV><FONT face=3DArial size=3D2></FONT> </DIV>
<DIV><FONT face=3DArial size=3D2>Regards,</FONT></DIV>
<DIV><FONT face=3DArial size=3D2>Bashir</FONT></DIV></BODY></HTML>

------=_NextPart_001_002C_01C31DF9.7CF57870--

------=_NextPart_000_002B_01C31DF9.7CF57870
Content-Type: application/msword;
name="KeyboardShortcuts.doc"
Content-Transfer-Encoding: base64
Content-Disposition: attachment;
filename="KeyboardShortcuts.doc"

0M8R4KGxGuEAAAAAAAAAAAAAAAAAAAAAPgADAP7/CQAGAAAAAAAAAAAAAACAAAAoQAAA
EAAAowAAAAEAAAD+////AAAAAJ8AAACgAAAA//////////////////////////////////////
//deleted
AAAAAAAAogEAAAAAACiAQAAAAAAKIBAAAAAAAAogEAABQAAAAAAAAAAAALYBAAAAAAA
AAAAAAB+LAAAAAAAAH4sAAAAAAAAfiwAACwAAACqLAAADAIAALYBAAAAAAAETYAAO4AAAD

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAA=

------=_NextPart_000_002B_01C31DF9.7CF57870
Content-Type: image/gif;
      name="doc1.gif"
Content-Transfer-Encoding: base64
Content-Disposition: attachment;
      filename="doc1.gif"

R0lGODlhlgBYALMAAABSSgBjWgBrUgBrYwBzUgB7YwhjYwhrYwh7YxBaUiFrY1KllKXOzt739//3
9////ywAAAAAlgBYAAAE/lDISau9OOvNu/9gKI5kaZ5oqq5s675wLM90bd94ru987//AoHBILBqP
//deleted
YC4EYZGnaLSBDQNYxi9QO+Id3DXMmFCVqEQBBjK0VsnHjbOc42yM5s7kQGKABndhiOS5mVm2
XwMtNcO8LtjKVs4Bdz/EMWS4UA2wjbSkJ03pSlv60pjOtKY3zelOe/rToA61qEdN6lKb+tScjgAA
ADs=

------=_NextPart_000_002B_01C31DF9.7CF57870
Content-Type: image/gif;
      name="graph1.gif"
Content-Transfer-Encoding: base64
Content-Disposition: attachment;
      filename="graph1.gif"

R0lGODlhZgHdAPcAAAQCBISChERCRMTCxCQiJKSipGRiZOTi5BQSFJSSlFRSVNTS1DQyNLSytHRy
//deleted
0lcuD2gBQzZQAgB+YHsP+C/4LsDVCTSASzGIwKY0IIHLwc0FXquAZH1qkB3QYKD1vTBxHHWCL
AwGwgAZ4KVCqGdnABwkogYJCgIIXYrjF1XGmgGdQgxBYQHgEQAEOCPA50rq4NgEBADs=

------=_NextPart_000_002B_01C31DF9.7CF57870--
```

2. MIME Parser

To process a MIME message, a mail reader needs to parse through the message and extract the components based on MIME headers contained in the message.

The following is an attempt to implement a simple MIME parser. The implementation is not exhaustive, but it will work in most basic cases.

```
using System;
using System.IO;
using System.Text;
using System.Web.Mail;

public    class Attachment {
     string filename = "none";
     byte[] content;
```

```
     public string FileName {
          get {return filename;}
          set {filename = value;}
     }

     public byte[] Content {
          get {return content;}
          set {content = value;}
     }
}

public class MimeParser {

     private MailMessage message = null;
     private StreamReader reader = null;
     private bool hasMorePart = false;

    //mime info fields
     private string mainBoundary = "";
     private string mimeVersion = "";
     private string mainContentType = "";
     private string mainHeader = "";

     public MailMessage MainMessage {
          get {return message;}
     }

     public bool HasMoreAttachment() {
          return hasMorePart;
     }

     public MimeParser(StreamReader input) {
          this.reader = input;
          GetMimeInfo();//main header, mime version, & main type
          message = GetMainMessage();
     }

     private void GetMimeInfo() {
          mainHeader = GetHeader();
          mimeVersion = GetHeaderValue(mainHeader, "mime-version");
          GetContentTypeAndBoundary(mainHeader, out mainContentType, out
mainBoundary);

          if (mainBoundary.Length != 0)
               MoveToPart(mainBoundary);
     }

     //finds the header part
     private String GetHeader() {
          StringBuilder sb = new StringBuilder();
          String s = null;
          do {
```

```
                s = reader.ReadLine();
                if (s != null && s.Length != 0)
                     sb.Append(s+"\r\n");
           } while (s != null && s.Length != 0);

           return sb.ToString();
     }

     private string GetHeaderValue(string header, string element) {
           string value;
           if (!element.EndsWith(":"))
                element = element + ":";

           string lowercaseHeader = header.ToLower();
           int elementIndex = lowercaseHeader.IndexOf(element);
           if (elementIndex < 0)
                return null;
           else {
                int colonIndex = lowercaseHeader.IndexOf(":", elementIndex);
                int crlfIndex = lowercaseHeader.IndexOf("\r\n", colonIndex);

                value = header.Substring(colonIndex+1, crlfIndex - colonIndex
-1).Trim();
                //in case there is folding
                int lastCrlfIndex;
                while (crlfIndex + 1 < header.Length -1 &&
char.IsWhiteSpace(header[crlfIndex+2])) {
                     lastCrlfIndex = crlfIndex;
                     crlfIndex =lowercaseHeader.IndexOf("\r\n",
lastCrlfIndex+2);
                     value +=  header.Substring(lastCrlfIndex+2, crlfIndex -
lastCrlfIndex -1).Trim();
                }
                return value;
           }
     }

     private void GetContentTypeAndBoundary(string header, out string
contentType, out string boundary) {
           string s = GetHeaderValue(header, "content-type");
           int semiColonIndex = s.IndexOf(";");
           boundary = "";
           if (semiColonIndex < 0)
                contentType = s;
           else {
                contentType = s.Substring(0, semiColonIndex);

                int boundaryIndex = s.ToLower().IndexOf("boundary",
semiColonIndex);
                if (boundaryIndex > 0) {
                     semiColonIndex = s.IndexOf(";", boundaryIndex);
                     if (semiColonIndex < 0)
                          boundary = s.Substring(boundaryIndex+9).Trim();
```

```
                    else
                         boundary=s.Substring(boundaryIndex+9,semiColonIndex-
boundaryIndex-9);

                    if (boundary.StartsWith("\""))
                         boundary =
boundary.Substring(boundary.IndexOf('\"')+1);
                    if (boundary.EndsWith("\""))
                         boundary = boundary.Substring(0, boundary.Length-1);
               }
          }
     }

     private MailMessage GetMainMessage() {
          MailMessage message = new MailMessage();
          message.To = GetHeaderValue(mainHeader, "to");
          message.From = GetHeaderValue(mainHeader,"from");
          message.Cc = GetHeaderValue(mainHeader, "cc");
          message.Subject = GetHeaderValue(mainHeader, "subject");

          if (mainBoundary.Length != 0)
               message.Body =
Encoding.ASCII.GetString(NextAttachment().Content);
          else {
              message.Body = reader.ReadToEnd();
                hasMorePart = false;
          }
          return message;
     }

     public Attachment NextAttachment() {
          if (! HasMoreAttachment())
               return null;

          Attachment attach = new Attachment();
          string header = GetHeader();

          string content="", bound="";
          GetContentTypeAndBoundary(header, out content, out bound);
          if (content.ToLower() == "multipart/alternative") {
               MoveToPart(bound);
               SkipToText(bound);
               attach.Content = Encoding.ASCII.GetBytes(GetData(bound,
false));
               SkipToEnd(bound);
          }
          else if (content.ToLower() == "multipart/mixed") {
              attach.Content = Encoding.ASCII.GetBytes("Nested Message Not
implemented");
               SkipToEnd(bound);
          }
          else if (IsBinary(content)) {
               attach.FileName = GetFileName(header);
```

```
                string encoding =  GetHeaderValue(header, "content-transfer-
encoding");
                if (encoding.ToLower()== "base64" ) {
                     string s = GetData(mainBoundary, true);
                     attach.Content = Convert.FromBase64String(s);
                }
                else
                     attach.Content =
Encoding.ASCII.GetBytes(GetData(mainBoundary, false));
           }
           else if (content.ToLower().IndexOf("text") >= 0) {
                attach.Content =
Encoding.ASCII.GetBytes(GetData(mainBoundary, false));
           }
           return attach;
     }

     private void MoveToPart(string boundary) {
           string line = reader.ReadLine();
           while (line.IndexOf(boundary) < 0)
                line = reader.ReadLine();

           if (line.IndexOf(boundary+"--") > 0)
                hasMorePart = false;
           else
                hasMorePart = true;

     }

     private bool IsBinary(string contentType) {
           string s = contentType.ToLower();
           return (s.IndexOf("application") >= 0) ||
                  (s.IndexOf("image") >= 0) ||
                  (s.IndexOf("audio") >= 0) ||
                  (s.IndexOf("video") >= 0);
     }

     private string GetFileName(string header) {
           int index = header.ToLower().IndexOf("filename=");
           string filename = header.Substring(index+9);
           if (filename.StartsWith("\""))
                filename = filename.Substring(filename.IndexOf('\"')+1);
           index = filename.IndexOf('\"');
           if (index > 0)
                filename = filename.Substring(0, index);

           return filename;
     }

     private string GetData(string boundary, bool binary) {
           StringBuilder sb = new StringBuilder();
           String s = "";
           int boundaryIndex;
```

```
            do {
                 s = reader.ReadLine();
                 boundaryIndex = s.IndexOf(boundary);
                 if (boundaryIndex < 0)
                      if (binary && s.Length > 0) {
                           sb.Append(s);
                      }
                      else
                          sb.Append(s+"\r\n");
            } while (boundaryIndex < 0);

            if (s.IndexOf(boundary+"--") > 0)
                 hasMorePart = false;
            else
                 hasMorePart = true;

            return sb.ToString();
       }

      //skip to the text/plain component of a multipart/alternative part
       private void SkipToText(string bound) {
            string header;
            do {
                 header = GetHeader();
                 if (header.ToLower().IndexOf("text/plain") < 0)
                           SkipBody(bound);
            } while (header.ToLower().IndexOf("text/plain") < 0);
       }

      //skip the body of this nested part
       private void SkipBody(string boundary) {
            string line = reader.ReadLine();
            while (line.IndexOf(boundary) < 0)
                 line = reader.ReadLine();
       }

      //skip to the end of this nested part
       private void SkipToEnd(string boundary) {
            string line = reader.ReadLine();
            while (line.IndexOf(boundary+"--") < 0)
                 line = reader.ReadLine();

            MoveToPart(mainBoundary);
       }
}
```

Note:

To use the MIME parser above, you need to follow the following steps:

- Connect to a POP server and obtain a socket,

- Use the socket to create StreamReader and StreamWriter objects
- Use the StreamWriter to issue a RETR command to retrieve a particular message and read the response indicator line.
- Use the StreamReader object to create an instance of MimeParser.
- Finally, use the following public property and methods of the MimeParser instance shown below to obtain the Message and the Attachments that may be contained in the message.

```
public MailMessage MainMessage
public bool HasMoreAttachment()
public Attachment NextAttachment()
```